

P1039R0

I got you, FAM - Flexible Array Members for C++

Published Proposal, 2018-10-19

Authors:

[JeanHeyd Meneide](#)

Arvid Gerstmann

Nicole Mazzuca

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Latest:

https://thephd.github.io/vendor/future_cxx/blob/master/papers/d1039.html

Reply To:

[JeanHeyd Meneide](#) | [@thephantomderp](#)

Abstract

Bringing C99 Flexible Array Member (FAM) syntax into C++ in a safe, well-defined manner.

Table of Contents

1 Revision History

1.1 Revision 0 - November 26th, 2018

2 Motivation

2.1 Contiguous Fixed Header and Data

2.2 Variable Length Arrays?

2.3 Runtime Sized Classes?

2.4 Places of Application

3 Design

3.1 Features and Explicit Opt-in

3.1.1 Feature: creation of FAM types is limited to certain expressions

3.1.2 Feature: Traits and Types

3.1.3 Feature: C Compatibility

3.1.4 Feature: Special Members

3.1.4.1 Special Member: Constructor of `std::fam_size`

3.1.4.2 Destructor

3.1.4.3 Copy/Move Constructor and Assignment

3.2 Examples

3.2.1 Simple: Trivial

3.2.2 Simple: Non-Trivial

3.2.3 Specialized Traits

3.2.4 UDP Packets

4 Proposed Wording

4.1 Proposed Feature Test Macro and Header

4.2 Intent

4.3 Synopsis

5 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0 - November 26th, 2018

Initial release.

§ 2. Motivation

C99 defined Flexible Array Members (FAMs), a way of having a dynamic array of similarly typed data laid out in a contiguous, flat format beyond an initial sequence in a struct. Flexible Array Members have been used to great success to interface with many low-latency networking and binary protocols and formats, making it the go-to choice for working with large amounts of data prefixed by header values. The key in its success in high-performance systems, components and software such as Operating Systems, Financial Trading and Tracking, Networking, and Firmware is the guarantee that there will be at most one allocation for a variably-sized structure.

Despite this success, nothing like this exists in C++, making its usage unspecified behavior by omission. All compilers warn or error about its usage when working with C from C++ that uses it, making it hard to confidently employ the C technique in C++ code bases. This presents a fairly painful chasm between what is possible in C and what is possible in C++, and prevents Bjarne Stroustrup's earliest vision of making it possible to fully cover and subsume all of C with C++.

This paper proposes a safe, well-reasoned extension of the C++ Language in order to accommodate and properly define FAMs for C++ that is compatible with the C99 standard.

§ 2.1. Contiguous Fixed Header and Data

There are many data structures in program, on disk, and on wire that are a fixed header plus a chunk of variably sized data. C++ has no way to represent this in a single allocation without the use of `char` buffers or `std::aligned_storage` plus a good, hefty helping of type punning/`reinterpret_cast`. FAM types guarantee not only contiguous layout, but at most one allocation to have all of the data required. This makes them ideal in high-performance environments where multiple allocations for a single packet of data are unacceptable, and usable in the other scenarios without locking out usability and performance benefits.

§ 2.2. Variable Length Arrays?

This proposal is **not** for variable length arrays. It does **not** cover all of the wants or needs that the previously proposed [\[n3662\]](#) does. Variable Length Arrays also do not cover all of the things Flexible Array Members do. Notably, Flexible Array Members only cover the case where memory is explicitly allocated for it already. Variable Length Arrays also do not accommodate heterogeneous data in their structure. This FAM specification does not include allowing anything but 0-sized default construction of a FAM type when an automatic storage duration value is created. This restriction may be lifted at another time in another revision, or in another paper entirely. This paper focuses exclusively on

covering the case of working with a pre-allocated buffer of space that the user has explicitly requested. (That pre-allocated buffer can be spelled `new my_fam_type(...)`).

§ 2.3. Runtime Sized Classes?

Classes of runtime size that also modified `sizeof()` were explored in Jeff Snyder and Richard Smith's [\[n4025\]](#). This proposal believes that the paper modified too much and went too far with what it changed about the core language. In particular, this paper instead focuses on the following:

- `sizeof()` will continue to always be well-formed and will never be ill-formed
- it does not attempt to cover or replace variable length arrays, which is a divisive topic currently unable to be solved
- it does not allow multiple or multiply-nested runtime classes

§ 2.4. Places of Application

FAMs are used in the wild for many binary formats, particularly those that find themselves need to precisely align and pack data according to transfer formats. It is also found in operating system and Some ripe places to use, and uses of, FAMs in the wild:

- Embedded and ARM code often use it to map regions of memory with prefixed data,
- Financial Bid and Transaction Protocols (such as OMX before reaching down into sub item headers, subscriber lists and set event lists),
- Heterogeneous Computing, such as [HSA BRIG modules](#),
- Networking protocols, such as
 - the ubiquitous lower layers such as [Internet Protocol \(IP\)](#), [Transmission Control Protocol \(TCP\)](#), and [User Datagram Protocol \(UDP\)](#), Link and similar in both userspace and kernel "ring0" space,
 - the increasingly popular [websockets|WebSockets],
- Operating System calls and system packages,
 - Windows in several locations, including [USN Journals](#) and [ReadDirectoryChangesW](#) return value,
 - all over the Linux Kernel (including [CGroups](#), variable string data, and more),

- High-performance storage solutions,
 - [Redis](#) makes extensive use of them
- LLVM and Clang,
 - Tail-allocated data structures are used frequently to save on the number of allocations even if they do not use the Flexible Array Member syntax directly,
- All over typical C code,
 - [Sanlock shared storage lock](#),
 - [\[spirv-cross\]](#),
- and, many, many more places.

This proposal's Flexible Array Members cover a good portion of the use cases found above.

§ 3. Design

In order to provide a reasonable feature set without having to compromise the entire language, FAMs

- implicitly mark their containing `structs/class`s as `final`,
- cannot be a sub-object of an array,
- must be the last member of their containing type,
- must have their containing type be the last member of any types it is used in,
- and, do not contribute to the `sizeof()` for the containing type, except for any padding necessary to reach the start of the flexible array member.

These sets of constraints help us properly integrate FAMs into the C++ language and properly matches the constraints of the C language. Our goals for proposing Flexible Array Members is simple:

1. Allow a succinct, portable way for C++ to refer to memory that is preceded by a header and laid out with its associated data payload in a contiguous manner.
2. Ease the porting of C code into C++ for users who wish to have correct, well-specified and well-defined behavior of their code.
3. Enable developers who must work with the data laid out in #1 to rely on standards-compliant, reasonable constructs in code.

Furthermore, this proposal aims to provide a set of overridable traits that -- if specified for a type -- will override how member count and data size are handled in C++. If the traits are not overridden, then the compiler is allowed to continue to use implementation-defined mechanisms for managing, controlling and deleting the memory.

These restrictions may seem overbearing, but they are for the good of the features laid out below. This proposal's restrictions are also forward-compatible: they can be relaxed or enhanced later without breaking old code, much like how `constexpr` was initially constrained and then generally relaxed later on once the power of the feature was fully understood.

§ 3.1. Features and Explicit Opt-in

Much of Flexible Array Member use will be coming from C code. It is imperative this code is well-formed, even when compiled as C++. Therefore, many of the operations that Flexible Array Members for C++ utilize must be completely defaulted. Consider the following simple FAM:

```
struct simple_fam_t {  
    int data[];  
};
```

This is valid code and under this proposal will continue to remain valid, without modification. This proposal does so through the application of the following features below.

§ 3.1.1. Feature: creation of FAM types is limited to certain expressions

FAM types can be created with `new`, but may not be used with array `new`. They can also be created with placement new.

When used as an automatic storage variable (e.g. "on the stack"), the flexible array member types just have a size that includes the non-FAM members, plus any padding necessary to get to the start of the FAM. This proposal allows automatic storage duration versions of the type as it mimics exactly how C handles it: a struct with an empty Flexible Array Member.

For the heap, FAMs are also still analogous with C. When allocated on the heap with `malloc` in C, the user makes room for it explicitly then performs type-punning of the returned `void*` data. In C++, this proposal would allow actually putting data in this type through the use of `new` as a proper analogy.

This matches C and also prevents a large myriad of cases where the type's boundaries are not clear and would violate invariants. For example, in the case of `new simple_fam_t[20];`, it is impossible to know where one flexible array member begins and the next ends without some sort of serious book keeping. The only allowed version of creating a flexible array member is with `new some_fam_type(...);`.

§ 3.1.2. Feature: Traits and Types

The proposed Flexible Array Members in C++ will feature a set of traits a user can override for their user-defined type. It also helps anyone thinking about Flexible Array Members for C++ to visualize the trait, type and functions. There are 3 traits and 1 type contained in the header `<fam>` and `<type_traits>`. One of them is overridable. Here is the full set of traits:

```
// header <fam>

namespace std {
    struct fam_size {
        fam_size(std::size_t element_count = 0) noexcept : n(element_count) {}

        std::size_t size() const noexcept {
            return n;
        }

    private:
        std::size_t n;
    };

    template <typename T>
    struct fam_traits {
        constexpr static std::size_t size (const T&) noexcept;
    }
}
```

```

// additions to
// header <type_traits>

namespace std {
    template <typename T>
    struct is_fam : std::integral_constant<
        bool, /* compiler intrinsic here */
    > {};

    template <typename T>
    using is_fam_v = is_fam<T>::value;

    template <typename T>
    struct fam_element {
        using type = /* compiler intrinsic here */;
    };

    template <typename T>
    using fam_element_t = fam_element<T>::type;
}

```

The various type queries here help programmers know if a type is a flexible array member and get the element of that type. The user can also use the traits to query the number of elements of a flexible array member for a given user-defined type with which this information is overridden. For example, consider the following:


```

#include <fam>
#include <cstdint>

struct id_list {
    std::size_t len;
    int64_t ids[];

    id_list(std::fam_size fs) : len(fs.size()) {}
};

namespace std {
    template <>
    struct fam_traits<id_list> {
        constexpr static size_type size (const id_list& il) noexcept {
            return il.len;
        }
    }
}

```

What this represents is a contract between the user and the C++ implementation. You are telling the implementation that you already manage and store the size yourself: thusly, the implementation knows to not bother storing any information about the number of elements, because those users are promising to construct and initialize `len` with the proper length and to book keep the number of created elements. This is relevant for both the automatically generated constructors and destructor in [§3.1.4 Feature: Special Members](#).

§ 3.1.3. Feature: C Compatibility

The size reported by the `fam_traits<T>::size(const T&)` static function can be greater than or equal to the number of elements actually used for any type `T` where `std::is_trivial_v<T>` evaluates to `true`.

"What in the world...?" Is what some people will say upon this realization, but there is an important point here. Consider the following absolutely confirming C implementation: the user asks for an object with a FAM `int[]`, requiring 20 `int`s worth of space: an implementation can give the user space for 30 `int`s. Furthermore, any trivial type (every type `T` from C has `std::is_trivial_v<T>` evaluate to true) does not need destructors run, so all the system needs to do is reclaim the memory. Therefore, most implementations store only the amount of memory allocated, not the number of elements.

Therefore, for any type which is trivial, an unspecialized `fam_traits<T>::size()` is not required to report exactly the number of elements the user asked for the FAM object, just a value greater than or equal to. This is because a valid compiler implementation of `size()` can be `return _Libc_allocated_size(fam_obj) / sizeof(fam_element_t<T>;`.

Note that this is only for trivial types, and is only mandated for perfect backwards compatibility with C code! For any FAM type that is non-trivial or whose array elements are non-trivial, the reported `size()` must be exactly equal to the number of successfully constructed elements so that the destructor can be run properly. The only reason to provide `size()` is to allow the compiler to generate a destructor that properly deletes the number of elements on your behalf, or the user in a FAM type's destructor to use it to destruct the correct number of values. If the type is trivially destructible, the user should not need to invoke the destructor on each element individually to begin with.

§ 3.1.4. Feature: Special Members

FAMs have one special constructor that can be generated. It then has the usual copy and move constructors, as well as copy and move assignment operators. The destructor is also generated (or left empty if the entire class is trivial).

§ 3.1.4.1. Special Member: Constructor of `std::fam_size`

To construct a FAM type within the above restrictions, a constructor that takes an argument of `std::fam_size` as its first argument is required. It may have more arguments than this, but to be used with `new T(std::fam_size(), arg0, ..., argN)` expressions, a constructor present must take a `std::fam_size` as the first argument. If one is not provided and `std::fam_traits` has not been specialized, then one is generated as follows:

```

#include <fam>

struct my_fam_t {
    std::string s[];

    my_fam_t() : my_fam_t(std::fam_size(0)) {}

    my_fam_t(std::fam_size __fs) {
        using __size_type = decltype(std::fam_traits<my_fam_t>::size);
        using __elem_type = std::string;
        __size_type __constructed = 0;
        __elem_type* __elem_ptr = reinterpret_cast<__elem_type*>(this);
        __size_type __sz = static_cast<__size_type>(__fs.size());
        try {
            for (; __constructed < __sz; ++__constructed, ++__elem_ptr)
                if constexpr (std::is_trivially_constructible_v<__elem_type>)
                    // default-init
                else {
                    new (__elem_ptr) __elem_type();
                }
            if constexpr (not std::is_trivially_destructible_v<__elem_type>)
                // exposition only
                // update constructed size,
                // so destructor of member can run properly
                _Libc_stored_size(*this, __constructed);
        }
        catch (...) {
            for (--__elem_ptr; __constructed != 0; --__constructed)
                __elem_ptr->~__elem_type();
            // rethrow
            throw;
        }
        if constexpr (not std::is_trivially_destructible_v<__elem_type>)
            // exposition only
            // update constructed size,
            // so destructor can run properly
            _Libc_stored_size(*this, __constructed);
    }
}

```

```

    }
}

```

If `std::fam_traits` has been specialized, then the compiler will not generate this constructor for the type and the program is ill-formed. If there are other members in this type that are not default constructible, then this constructor will not be written for the type and the program will be ill-formed. The default, no-argument constructor will simply call the deferred constructor as if defined like `type_name() : type_name(std::fam_size(0)) {}`.

§ 3.1.4.2. Destructor

The destructor is also automatically generated for FAM types. An exemplary implementation is as follows:

```

struct my_fam_t {
    std::string s[];

    ~my_fam_t() {
        using __elem_type = std::string;
        using __size_type = decltype(
            std::fam_traits<my_fam_t>::size(
                std::declval<__elem_type>()
            )
        );
        if constexpr(std::is_trivially_destructible_v<__elem_type>)
            // no-op
        else {
            __size_type __sz = std::fam_traits<my_fam_t>::size(
                __elem_type* __elem_ptr = reinterpret_cast<__elem_t
                __elem_type* __elem_ptr_end = __elem_ptr;
                __elem_ptr += __sz;
                for (; __elem_ptr != __elem_ptr_end; --__elem_ptr)
                    __elem_ptr->~__elem_type();
            }
        }
    }
}

```

A user does not ever have to write a destructor for their FAM type: one will always be generated that is correct for dealing with all of the members, plus the flexible array member's data. When a FAM type is destructed, the elements of the FAM are destroyed in reverse order, and then the other elements of the class are destroyed as normal. See the [§3.2.2 Simple: Non-Trivial](#) for an example of the default construction and destruction orders.

§ 3.1.4.3. *Copy/Move Constructor and Assignment*

Copy/move constructors and copy/move assignment all follow the same pattern as the constructor and destructor in this case. They are seen as if invoking a constructor with `std::fam_size`, and then copy/move constructing each element of the array (that is, it performs a by-value copy of each element). The sizes of both FAMs will compare equal after copy and move operations (modulo any insane `std::fam_traits` specializations).

§ 3.2. Examples

Here are some examples of the syntax and some of its invariants.

§ 3.2.1. **Simple: Trivial**

Here is a very simple FAM type:

```

#include <fam>

struct easy_fam_t {
    int ids[];
};

#include <cassert>

int main () {
    using my_traits = std::fam_traits<easy_fam_t>;

    easy_fam_t automatic_fam_object;
    std::size_t automatic_len
        = my_traits::size(automatic_fam_object);
    assert(automatic_len == 0);
    // following is ill-formed:
    // fam_t other(std::fam_size(1));
    // error: cannot create flexible array
    // member of varying size in automatic storage

    easy_fam_t* dynamic_fam_object_raw = new easy_fam_t(std::fam_size(2));
    std::size_t dynamic_raw_len
        = my_traits::size(*dynamic_fam_object_raw);
    // !! IMPORTANT
    // reported raw size can be
    // 24, or GREATER than for trivial types!!
    assert(dynamic_raw_len >= 24);
    delete dynamic_fam_object;

    return 0;
}

```

§ 3.2.2. Simple: Non-Trivial

Here is a FAM type with a non-trivial element that prints its id on construction and destruction:

```

#include <iostream>
#include <fam>

struct tracer {
    static int x;

    int id = ++x;

    tracer() {
        std::cout << "constructed " << id << std::endl;
    }

    ~tracer() {
        std::cout << "destructured " << id << std::endl;
    }
};

int tracer::x = 0;

struct tracing_fam_t {
    tracer tracers[];
};

int main () {
    using my_traits = std::fam_traits<tracing_fam_t>;

    std::unique_ptr<tracing_fam_t> dynamic_fam_object
        = std::make_unique<tracing_fam_t>(std::fam_size(5));
    std::size_t dynamic_len
        = my_traits::size(*dynamic_fam_object);
    assert(dynamic_raw_len == 5);

    return 0;
}

```

This will print:

```
constructed 1  
constructed 2  
constructed 3  
constructed 4  
constructed 5  
destruxted 5  
destruxted 4  
destruxted 3  
destruxted 2  
destruxted 1
```

Note that arrays are constructed in forward-linear order by default, and destructs in reverse-linear order by default. If a user wants to override the constructor or destructor to behave differently, they are more than welcome to change the semantics of destruction order.

§ 3.2.3. Specialized Traits

The below example shows off a specialized traits class with a custom constructor. Note that in this example there is no destructor definition: it is generated for us and properly uses `std::fam_traits<...>::size(obj);` to get the element count to destroy.


```

#include <iostream>
#include <fam>

struct tracer_2 {
    static int x;

    int id;

    tracer_2(int id_boost) : id(++x + id_boost) {
        std::cout << "constructed " << id << std::endl;
    }

    ~tracer_2() {
        std::cout << "destruced " << id << std::endl;
    }
};

int tracer_2::x = 0;

struct tracing_fam_2_t {
    int len;
    tracer_2 tracers[];

    tracing_fam_2_t(std::fam_size fs, int id_boost) : len(fs.size()) {
        std::cout << "manually constructing fam type" << std::endl;
        for (int i = 0; i < len; ++i) {
            new (&tracers[i]) tracer_2(id_boost);
        }
    }
};

template <>
struct fam_traits {
    constexpr static int size(const tracing_fam_2_t& tf2) noexcept {
        return tf2.len;
    }
};

int main () {
    using my_traits = std::fam_traits<tracing_fam_2_t>;

    std::unique_ptr<tracing_fam_2_t> dynamic_fam_object
        = std::make_unique<tracing_fam_2_t>(std::fam_size(3), 2);
    std::size_t dynamic_len

```

```

        = my_traits::size(*dynamic_fam_object);
assert(dynamic_fam_object->len == dynamic_raw_len);
assert(dynamic_fam_object->len == 3);

// ill-formed
// tracing_fam_2_t auto_tf2;
// error: cannot call default
// constructor tracing_fam_2_t(std::fam_size(0))

return 0;
}

```

This will print:

```

manually constructing fam type
constructed 3
constructed 4
constructed 5
deconstructed 5
deconstructed 4
deconstructed 3

```

Note here that `tracing_fam_2_t` has more than just `std::fam_size` as an argument for its special constructor. Therefore, a generated default constructor that takes 0 arguments and defers to a constructor as if `tracing_fam_2_t() : tracing_fam_2_t(std::fam_size(0))` is ill-formed. Therefore, it is not generated and attempt to default-construct this type would result in an error.

§ 3.2.4. UDP Packets

This extension is also a natural fit for many on-the-wire and on-disk data types. Here is an example for UDP packets.

```

#include <cstdint>
#include <cstdint>
#include <fam>

struct udp {
    std::uint16_t source_port;
    std::uint16_t destination_port;
    std::uint16_t data_size;
    std::uint16_t checksum;
    std::byte      data[];

    udp(std::fam_size fs)
        : source_port(0),
          destination_port(0),
          data_size(static_cast<std::uint16_t>(fs.size())),
          checksum(0) {}

    int packet_size() const {
        static_assert((sizeof(udp) * CHAR_BIT) == (8 * 8),
            "compiler did not lay out 4 uint16_t's in exactly
            return static_cast<int>(data_size + 8);
    }

    const char* packet_data () const {
        return reinterpret_cast<const char*>(this);
    }

    char* packet_data () {
        return reinterpret_cast<char*>(this);
    }
};

template <>
struct fam_traits<udp> {
    constexpr static std::uint16_t size (const udp& u) {
        return u.data_size;
    }
};

#include<iostream>
#include<cstring>
#include<cstdlib>

```

```

#include<arpa/inet.h>
#include<sys/socket.h>

typedef struct sockaddr base_socket_address;
typedef struct sockaddr_in socket_address;

struct socket_t {
    int handle;

    socket_t (int h) : handle(h) {}
    socket_t(std::nullptr_t) : handle(-1) {}
    operator int() const {
        return handle;
    }

    bool operator==(const socket_t& rhs) const {
        return handle == rhs.handle;
    }

    bool operator!=(const socket_t& rhs) const {
        return handle != rhs.handle;
    }

    bool operator==(std::nullptr_t) const {
        return handle == -1;
    }

    bool operator!=(std::nullptr_t) const {
        return handle != -1;
    }
};

struct socket_deleter {
    void operator()(socket_t s) const {
        close(s.handle);
    }
};

int main () {

    //create a UDP socket
    std::unique_ptr<socket_t, socket_deleter> s(
        socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP));
}

```

```

    if (!s) {
        std::cout << "Couldn't open a UDP socket!" << std::endl;
        return -1;
    }

    socket_address address_out;
    std::memset(reinterpret_cast<char*>(&address_out),
                0,
                sizeof(address_out));
    address_out.sin_family = AF_INET;
    address_out.sin_port = htons(8888);
    address_out.sin_addr.s_addr = htonl(3456);

    // create UDP packet of 460 bytes
    std::unique_ptr<udp> packet
        = std::make_unique<udp>(std::fam_size(460));
    /* Super Cool Serialization Here */
    // Send it over!
    int result = sendto(*s,
                        udp->packet_data(),
                        udp->packet_size(),
                        0,
                        reinterpret_cast<base_socket_address>(&address_out),
                        sizeof(address_out));
    if (result == -1) {
        std::cout << "Could not send a UDP packet!" << std::endl;
        return -1;
    }
    return 0;
}

```

Tail-allocated structures now become very easy to specify, creation of them saves on additional allocations and avoids type punning shenanigans, and can easily match programmer intent while keeping the strong type safety built into C++.

§ 4. Proposed Wording

The wording for this proposal is incomplete. The author is working on it, and suggestions as to what clauses should be modified are welcome!

Any attempts at wording would be relative to [\[n4762\]](#).

§ 4.1. Proposed Feature Test Macro and Header

The proposed feature test macro is `__cpp_flexible_array_members` with a value of 201811LL. The added header is `<fam>`, and the additional traits go in `<type_traits>` as specified.

§ 4.2. Intent

The intent of this wording is to create a new type in the C++ language called a Flexible Array Member (FAM) type (FAM type). FAM types:

- shall be implicitly final,
- shall have the flexible array member type come last,
- shall be of the form `type-identifier variable-name [ ]`,
- shall only be created with a variable size through placement new or new syntax,
- shall not be an object or sub-object of an array,
- shall have a special constructor that takes at least 1 argument of `std::fam_size` as the first argument with any number of additional arguments,
- shall either specialize `std::fam_traits` with the required `size` function and write a constructor following the above restriction or have one generated,
- may have a default constructor generated which defers to another constructor with only the argument `std::fam_size(0)`,
- shall not contribute to the `sizeof()` value for the type except for any necessary implementation-defined padding to reach the flexible array member,
- and if either the type of the flexible array member's element type evaluates `std::is_trivial<element_type>` or `std::fam_traits` is specialized by the program to true then the FAM type is not required to report the exact element count requested by the use of a `std::fam_size` constructor.

Additionally, the wording is meant to create 1 new type -- `std::fam_size` -- whose type references a compiler-defined type that is only named in the program through inclusion of the header `<fam>` with the type identifier `std::fam_size`. It will also created 1 new program-specializable trait -- `std::fam_traits` -- that is also available in the `<fam>` header. If the `std::fam_traits` template

type is not specialized, then it is only required that the implementation report exactly the element count passed in through `std::fam_size`, modulo the exception for trivial types above.

§ 4.3. Synopsis

```
// <fam>

namespace std {
    struct fam_size {
        fam_size(std::size_t element_count = 0) noexcept;
        std::size_t size() const noexcept;
    };

    template <typename T>
    struct fam_traits {
        constexpr static std::size_t size (const T&) noexcept;
    }
}
```

```
// <type_traits>

namespace std {
    // ...

    template <typename T>
    struct is_fam;

    template <typename T>
    using is_fam_v = is_fam<T>::value;

    template <typename T>
    struct fam_element;

    template <typename T>
    using fam_element_t = fam_element<T>::type;

    // ...
}
```

§ 5. Acknowledgements

A big thank you to Agustín Bergé for helping to hammer down this initial idea before the group of us spiraled off into the weeds!

Thank you to Simon Brand for providing a few example use cases and telling us about HSA BRIG modules.

Thank you to Matt Godbolt for his small chat during a C++Now dinner about the ways in which people use what are essentially Flexible Array Members with gratuitous type punning in High Frequency Trade network programs.

Thank you to Jeff Snyder for his insights with his previous papers and his wisdom sharing at C++Now, and Chandler Carruth for pointing me in his direction.

§ References

§ Informative References

[FILE_NOTIFY_INFORMATION]

Microsoft. [_FILE_NOTIFY_INFORMATION structure](https://docs.microsoft.com/en-us/windows/desktop/api/winnt/ns-winnt-_file_notify_information). September 27th, 2018. URL: https://docs.microsoft.com/en-us/windows/desktop/api/winnt/ns-winnt-_file_notify_information

[HSA-BRIG]

HSA Foundation. [HSA Specification Library: BRIG Modules](http://www.hsafoundation.com/html/Content/PRM/Topics/18_BRIG/BRIG_module.htm). 2015. URL: http://www.hsafoundation.com/html/Content/PRM/Topics/18_BRIG/BRIG_module.htm

[LINUX-CGROUPS]

Linus Torvalds et. al.. [Linux CGroups](https://github.com/torvalds/linux/blob/2ce7135adc9ad081aa3c49744144376ac74fea60/include/linux/cgroup-defs.h#L450). October 4th, 2018. URL: <https://github.com/torvalds/linux/blob/2ce7135adc9ad081aa3c49744144376ac74fea60/include/linux/cgroup-defs.h#L450>

[N3662]

Lawrence Crowl; Matt Austern. [C++ Dynamic Arrays](https://wg21.link/n3662). April 19th, 2013. URL: <https://wg21.link/n3662>

[N4025]

Jeff Snyder; Richard Smith. [C++ Dynamic Arrays](https://wg21.link/n4025). May 23rd, 2014. URL: <https://wg21.link/n4025>

[N4762]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. [N4762 - Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf). May 7th, 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf>

[REDIS]

Salvatore Sanfilippo; redislabs. [Redis Source](#). August 29th, 2018. URL: <https://github.com/antirez/sds/blob/master/sds.h#L47>

[SANLOCK-OCAML]

Si Beaumont. [OCaml ctypes: support for Flexible Array members](#). March 30th, 2016. URL: <http://simonjbeaumont.com/posts/ocaml-ctypes-flexible-array-member/>

[SPIRV-CROSS]

Khronos Group. [Flexible array member in CPP output](#). November 2nd, 2018. URL: <https://github.com/KhronosGroup/SPIRV-Cross/issues/10>

[TCP-IP]

Information Sciences Institute, University of Southern California. [Transmission Control Protocol](#). October 17th, 2018. URL: <https://tools.ietf.org/html/rfc793>

[UDP]

J. Postel; ISI. [User Datagram Protocol](#). August 28th, 1980. URL: <https://tools.ietf.org/html/rfc768>

[USN-RECORD-V4]

Microsoft. [USN Records](#). October 17th, 2018. URL: https://docs.microsoft.com/en-us/windows/desktop/api/winiocctl/ns-winiocctl-usn_record_v4